

LEARNING ROADMAP

QA & SDET Roadmap

From Manual Testing to Automation Engineering

Intensive

35-45 hrs/week

Balanced

15-20 hrs/week

Flexible

8-10 hrs/week

■ 7 Phases ★ Beginner to Advanced

Overview

Quality Assurance isn't about finding bugs – it's about preventing them. This roadmap takes you from understanding how to think like a tester to building automation frameworks that catch issues before users do. Whether you want to be a Manual QA, Automation Engineer, or SDET (Software Development Engineer in Test), this path builds the foundation. The SDET role originated at Microsoft and Amazon when they needed engineers who could code at developer level while thinking like testers. Today, companies pay premium salaries for people who can do both.

Who is this for?

Anyone interested in software quality – career changers, developers who want to understand testing, manual testers transitioning to automation, CS students exploring QA careers, or anyone who notices bugs everywhere and can't help but report them.

Prerequisites

- Basic computer literacy
- Curiosity about how software breaks
- Attention to detail (or willingness to develop it)
- No programming required to start (you'll learn)
- Patience – good testing is methodical

What you'll learn

- Think like a tester – find edge cases others miss
- Design test cases using proven techniques
- Write automation code in Python or JavaScript
- Build and maintain test automation frameworks
- Test APIs thoroughly (REST, GraphQL)
- Integrate tests into CI/CD pipelines
- Understand performance and security testing basics

- Communicate quality metrics to stakeholders

Learning Plans

Choose a plan that fits your schedule. All plans cover the same content.

Intensive

Fast Track

35-45

hrs/week

**350-
450**

total hrs

Full-time immersive learning. Ideal for career changers committed to becoming an SDET or Automation Engineer.

Ideal for:

- Career changers
- Bootcamp-style learners
- Those preparing for SDET interviews
- Recent graduates

Balanced

Recommended

15-20

hrs/week

**360-
480**

total hrs

Steady progress while working. Learn testing at work, automation on evenings/weekends. The recommended path.

Ideal for:

- Manual testers upskilling
- Working professionals
- Developers learning testing
- Self-paced learners

Flexible

Self-paced

8-10

hrs/week

**400-
500**

total hrs

Gradual skill building with time to practice deeply. Good for those who want thorough understanding.

Ideal for:

- Full-time employees
- Parents and caregivers
- Those learning multiple skills
- Weekend learners

Phase Schedule by Plan

Phase	Topic	Intensive	Balanced	Flexible
1	The Tester Mindset	Week 1	Weeks 1-2	Weeks 1-4
2	Test Design Techniques	Week 2	Weeks 3-4	Weeks 5-8
3	Programming for Testers	Weeks 3-4	Weeks 5-8	Weeks 9-16
4	Web UI Automation	Weeks 5-6	Weeks 9-12	Weeks 17-24
5	API Testing	Week 7	Weeks 13-16	Weeks 25-32
6	CI/CD & Test Infrastructure	Week 8	Weeks 17-20	Weeks 33-40
7	Specializations	Weeks 9-10	Weeks 21-26	Weeks 41-52

1 The Tester Mindset

Intensive: Week 1

Balanced: Weeks 1-2

Flexible: Weeks 1-4

Before learning tools, you need to learn how to think. Great testers don't just follow test cases – they question assumptions, explore edge cases, and think about what could go wrong. This mindset separates people who 'check if it works' from those who truly ensure quality. A tester mindset means: thinking critically instead of accepting, evaluating failure possibilities, and aiming to prevent bugs rather than just find them.

Learning Objectives

- Understand the difference between testing and checking
- Develop critical thinking and questioning skills
- Learn to see software from the user's perspective
- Understand the software development lifecycle (SDLC)
- Know different types of testing and when to use them
- Communicate defects clearly and effectively

Topics to Cover

What is Testing, Really?

WHY: Testing isn't clicking buttons to see if they work. It's investigating software to discover information that stakeholders need. Understanding this changes how you approach every test.

- Testing vs Checking
- Verification vs Validation
- Quality Assurance vs Quality Control
- The cost of bugs at different stages
- Why bugs escape to production
- Testing's role in the team

The Tester's Thinking Process

WHY: Good testers ask 'what if?' constantly. They challenge assumptions, consider edge cases, and think adversarially. This thinking is learnable.

- Critical thinking for testers
- Questioning assumptions
- Edge case identification
- Negative testing mindset
- Risk-based thinking
- Heuristics and oracles

Types of Testing

WHY: Different situations call for different approaches. Knowing when to use unit vs integration vs E2E testing prevents wasted effort and gaps in coverage.

- Unit, Integration, System, E2E
- Functional vs Non-functional
- Black-box vs White-box vs Grey-box
- Smoke vs Sanity vs Regression
- Alpha vs Beta testing
- The Testing Pyramid

SDLC & Testing in Agile

WHY: Modern teams are Agile. Understanding sprints, user stories, and definition of done lets you integrate testing throughout development, not just at the end.

- Waterfall vs Agile testing
- Shift-left testing
- Testing in Scrum
- User stories and acceptance criteria
- Definition of Done
- Continuous testing mindset

Bug Reporting That Gets Fixed

WHY: A bug report that developers can't reproduce gets ignored. Clear, reproducible bug reports with proper severity and priority get fixed fast.

- Writing clear bug reports
- Steps to reproduce
- Expected vs Actual results
- Severity vs Priority
- Bug lifecycle
- Screenshots and evidence

Hands-on Projects

Explore and Break an App

Pick any web application (e-commerce site, social media, etc.). Spend 2 hours exploring it with the goal of finding bugs. Document at least 10 issues with proper bug reports. Focus on edge cases, error handling, and unexpected inputs.

Exploratory testing

Bug reporting

Critical thinking

Requirements Analysis Exercise

Take a simple feature spec (login page, shopping cart). Before any testing, write down all the questions you'd ask the product owner. What's ambiguous? What edge cases aren't mentioned? What could go wrong?

Requirements analysis

Critical thinking

Communication

Resources

BOOK [Lessons Learned in Software Testing](#)

COMMUNITY [Ministry of Testing](#)

CERTIFICATION [ISTQB Foundation Syllabus](#)

Milestone



Find bugs in a production application that real users experience. Write bug reports that a developer would thank you for. Articulate why testing is more than 'making sure it works.'

2 Test Design Techniques

Intensive: Week 2

Balanced: Weeks 3-4

Flexible: Weeks 5-8

You can't test everything – there are infinite possible inputs and scenarios. Test design techniques help you choose the right tests that maximize coverage with minimum effort. These techniques are used by both manual and automation testers. Master them, and you'll find bugs others miss while writing fewer tests.

Learning Objectives

- Apply equivalence partitioning to reduce test cases
- Use boundary value analysis to find edge case bugs
- Create decision tables for complex logic
- Design state transition tests for workflows
- Use error guessing based on experience
- Conduct effective exploratory testing sessions

Topics to Cover

Equivalence Partitioning

WHY: You can't test every possible input. Equivalence partitioning divides inputs into groups where values should behave the same. Test one from each group – if it works, others in that group should too.

- Valid vs Invalid partitions
- Identifying partitions
- Selecting representative values
- Combining with other techniques
- Practical examples
- Common mistakes

Boundary Value Analysis

WHY: Bugs love boundaries. Off-by-one errors are everywhere. Testing at 0, 1, max-1, max, max+1 catches bugs that testing '5' or '50' would miss.

- Why boundaries matter
- Identifying boundary values
- Two-value vs Three-value BVA
- Combining with equivalence partitioning
- Date and time boundaries
- Array and string boundaries

Decision Table Testing

WHY: Complex business rules have combinations of conditions. Decision tables ensure you test all relevant combinations without missing any – or wasting time on impossible ones.

- Conditions and actions
- Building decision tables
- Reducing combinations
- When to use decision tables
- Practical examples
- Automating from decision tables

State Transition Testing

WHY: Many features have states (draft → submitted → approved → published). Testing transitions – including invalid ones – catches bugs in workflow logic.

- State diagrams
- Valid transitions
- Invalid transition testing
- State coverage criteria
- Real-world examples
- E-commerce checkout flows

Exploratory Testing

WHY: Scripted tests find expected bugs. Exploratory testing – where you simultaneously learn, design, and execute tests – finds the unexpected. Senior testers rely on it heavily.

- Charter-based exploration
- Time-boxed sessions
- Note-taking techniques
- When to use exploratory testing
- Session-based test management
- Combining with automation

Hands-on Projects

E-commerce Test Suite Design

Design test cases for an e-commerce checkout: cart → shipping → payment → confirmation. Apply all techniques: equivalence partitioning for payment amounts, BVA for quantities, decision tables for shipping options, state transitions for order status.

All test design techniques

Test case documentation

Prioritization

Exploratory Testing Charter

Create and execute a 60-minute exploratory testing session on a web app. Write a charter, take notes, log bugs found, and write a debrief report. Compare bugs found to what scripted tests would catch.

Exploratory testing

Session management

Documentation

Resources

BOOK [Software Testing Techniques \(Boris Beizer\)](#)

TUTORIAL [Test Design Techniques - Guru99](#)

BOOK [Explore It! \(Elisabeth Hendrickson\)](#)

Milestone



Given any feature, systematically derive test cases using appropriate techniques. Explain why you chose certain tests and what risks they mitigate. Conduct an exploratory session that finds bugs scripted tests missed.

3 Programming for Testers

Intensive: Weeks 3-4

Balanced: Weeks 5-8

Flexible: Weeks 9-16

Automation requires code. This phase teaches programming specifically for testing – not to become a developer, but to write reliable, maintainable test automation. We focus on Python (widely used, beginner-friendly) or JavaScript (essential for web testing). Choose one, get comfortable, then expand. You'll learn just enough to automate, not to build applications.

Learning Objectives

- Write basic programs in Python or JavaScript
- Understand OOP concepts needed for test frameworks
- Work with files, JSON, and APIs programmatically
- Use version control (Git) for test code
- Write clean, maintainable test code
- Debug code effectively

Topics to Cover

Programming Fundamentals

WHY: Test automation is programming. Without understanding variables, loops, and functions, you'll be copying code without understanding it – and unable to debug when it breaks.

- Variables and data types
- Conditionals (if/else)
- Loops (for, while)
- Functions
- Error handling (try/catch)
- Data structures (lists, dictionaries)

Object-Oriented Programming

WHY: Test frameworks use OOP heavily. Page Object Model, the standard pattern for UI automation, requires understanding classes, objects, and inheritance.

- Classes and objects
- Methods and properties
- Inheritance basics
- Why OOP for testing
- Page Object Model introduction
- Practical OOP patterns

Working with Data

WHY: Tests need data — from files, APIs, and databases. Knowing how to read JSON, parse CSVs, and query databases makes your tests data-driven and flexible.

- Reading/writing files
- JSON parsing
- CSV handling
- Environment variables
- Test data management
- Data-driven testing basics

Git for Test Code

WHY: Test code is code. It needs version control, code review, and collaboration. Teams that don't version their tests have chaos.

- Git basics for testers
- Branching for test development
- Pull requests for test code
- Handling merge conflicts
- .gitignore for test artifacts
- Collaborating with developers

Clean Test Code

WHY: Tests that are hard to read are hard to maintain. Teams spend 30-40% of QA time maintaining test suites. Clean code reduces that dramatically.

- Naming conventions
- Single responsibility
- DRY principle
- Readable assertions
- Comments and documentation
- Code review for tests

Hands-on Projects

API Data Processor

Write a script that: calls a public API, parses the JSON response, filters data based on criteria, and writes results to a file. Practice error handling for network failures.

HTTP requests

JSON parsing

File I/O

Error handling

Test Data Generator

Build a utility that generates test data: random users, addresses, orders. Use functions and classes to organize code. Output to JSON files that your tests can use.

OOP

Data structures

File handling

Code organization

Resources

BOOK

[Automate the Boring Stuff with Python](#)

TUTORIAL

[JavaScript.info](#)

COURSE

[Python for Testers \(Pluralsight\)](#)

Milestone



Write a program that fetches data from an API, processes it, and outputs results – without copying code. Debug errors independently. Understand every line of code in your test files.

4

Web UI Automation

Intensive: Weeks 5-6

Balanced: Weeks 9-12

Flexible: Weeks 17-24

Now you automate browser testing. This phase covers Playwright (modern, fast, recommended) and Selenium (legacy, still widely used). You'll learn to interact with web elements, handle waits, and structure tests using Page Object Model. The goal: reliable tests that don't flake, run fast, and are easy to maintain.

Learning Objectives

- Automate browser interactions with Playwright or Selenium
- Write reliable locators that don't break
- Handle waits and asynchronous behavior
- Implement Page Object Model
- Debug flaky tests
- Run tests in parallel and across browsers

Topics to Cover

Playwright Fundamentals

WHY: Playwright is the modern choice – faster than Selenium, auto-waits built-in, better debugging. Microsoft built it after learning from Selenium's limitations. New projects should start here.

- Installation and setup
- Browser contexts and pages
- Locators (getByRole, getByText)
- Actions (click, fill, select)
- Assertions
- Screenshots and traces
- Playwright Inspector

Selenium WebDriver

WHY: Selenium has 22% market share and huge existing codebases. Many jobs require it. Understanding Selenium also helps when maintaining legacy suites.

- WebDriver architecture
- Browser drivers
- Locator strategies (ID, CSS, XPath)
- Explicit vs Implicit waits
- Actions and interactions
- Handling alerts and frames

Reliable Locators

WHY: Tests break when locators break. UI changes constantly. Writing robust locators — and knowing which to avoid — is the difference between flaky and reliable tests.

- Why locators break
- Accessibility-first locators
- Data-testid attributes
- CSS selectors deep dive
- XPath when necessary
- Avoiding brittle locators

Page Object Model

WHY: POM separates test logic from page structure. When the UI changes, you update one place instead of 50 tests. It's the industry standard for maintainable UI automation.

- Why POM exists
- Creating page classes
- Encapsulating elements and actions
- Reusable components
- Test data handling
- POM variations and patterns

Handling Flaky Tests

WHY: Flaky tests erode trust. Teams start ignoring failures. Understanding why tests flake — timing, state, environment — and how to fix them is a senior skill.

- Common causes of flakiness
- Race conditions
- Test isolation
- Retry strategies
- Debugging with traces
- When to delete vs fix

Hands-on Projects

E-commerce Test Suite

Automate end-to-end tests for an e-commerce site (use a demo site like Sauce Demo or Swag Labs). Cover: login, product search, add to cart, checkout. Use Page Object Model. Run across Chrome and Firefox.

Playwright/Selenium

POM

Cross-browser testing

Assertions

Form Validation Testing

Create a test suite for a complex form with validation. Test all boundary values, error messages, and edge cases (special characters, empty fields, too-long inputs). Make tests data-driven.

Data-driven testing

Validation testing

Test organization

Resources

DOCUMENTATION

[Playwright Documentation](#)

DOCUMENTATION

[Selenium Documentation](#)

COURSE

[Test Automation University](#)

Milestone



Build a test suite with 20+ tests using POM. Run them in CI. Have zero flaky tests. When the UI changes, update only the page object, not the tests.

5 API Testing

Intensive: Week 7

Balanced: Weeks 13-16

Flexible: Weeks 25-32

Most bugs hide below the UI. API testing is faster, more stable, and catches issues earlier. Modern applications are API-first – testing at this layer gives you more coverage with less effort. You'll learn both manual API testing (Postman) and automated API testing (RestAssured, Playwright, or requests library).

Learning Objectives

- Understand REST API architecture
- Test APIs manually using Postman
- Automate API tests in code
- Validate responses (status, body, headers)
- Test authentication flows
- Understand GraphQL testing basics

Topics to Cover

REST API Fundamentals

WHY: 92% of enterprise traffic flows through REST APIs. Understanding HTTP methods, status codes, and request/response structure is fundamental to API testing.

- HTTP methods (GET, POST, PUT, DELETE)
- Status codes and meaning
- Request/response structure
- Headers and authentication
- Query params vs body
- REST conventions

Postman for API Testing

WHY: Postman is the industry standard for API exploration and manual testing. Collections, environments, and test scripts make it powerful for both exploration and automation.

- Creating requests
- Collections and folders
- Environment variables
- Pre-request scripts
- Test scripts and assertions
- Running collections (Newman)

API Test Automation

WHY: Automated API tests run in CI/CD and catch regressions fast. They're more stable than UI tests and execute in milliseconds instead of seconds.

- RestAssured (Java)
- requests + pytest (Python)
- Playwright API testing
- Response validation
- Schema validation
- Chaining API calls

Authentication Testing

WHY: Auth bugs are security bugs. Testing login, tokens, session management, and authorization prevents vulnerabilities that could expose user data.

- Basic auth, API keys, Bearer tokens
- OAuth2 flows
- JWT validation
- Testing authorization (roles/permissions)
- Session management
- Security considerations

GraphQL Testing

WHY: GraphQL adoption is growing. It requires different testing approaches – queries, mutations, variables, and error handling work differently than REST.

- GraphQL vs REST
- Queries and mutations
- Variables and fragments
- Error handling
- Testing with Postman
- Automating GraphQL tests

Hands-on Projects

API Test Suite

Build a comprehensive test suite for a public API (GitHub, Spotify, or a practice API). Cover all CRUD operations, error handling, authentication, and edge cases. Achieve 80%+ endpoint coverage.

API automation Test organization Assertions
Data handling

Contract Testing Introduction

Set up contract tests between a mock frontend and backend API. Use Pact or similar tool. Verify that API changes don't break consumers.

Contract testing API contracts
Consumer-driven testing

Resources

DOCUMENTATION [Postman Learning Center](#)

DOCUMENTATION [RestAssured Documentation](#)

TUTORIAL [API Testing Best Practices](#)

Milestone



Test any REST API confidently with Postman. Automate API tests that run in CI. Explain why API tests should form the bulk of your automation (testing pyramid).

6

CI/CD & Test Infrastructure

Intensive: Week 8

Balanced: Weeks 17-20

Flexible: Weeks 33-40

Tests that don't run automatically don't run at all. This phase covers integrating tests into CI/CD pipelines, managing test environments, reporting results, and building the infrastructure that makes continuous testing possible. This is what transforms testing from a gate to a continuous feedback loop.

Learning Objectives

- Integrate tests into CI/CD pipelines
- Configure test environments and data
- Generate and analyze test reports
- Manage test artifacts
- Implement test selection and parallelization
- Monitor test health and flakiness

Topics to Cover

CI/CD for Testing

WHY: Shift-left testing means testing early and often. CI/CD pipelines run tests on every commit, catching bugs before they reach main. Understanding pipelines is essential for modern QA.

- CI/CD concepts
- GitHub Actions basics
- Jenkins fundamentals
- Triggering tests on commits
- Pipeline stages
- Parallel test execution

Test Environments

WHY: Tests need consistent environments. Docker, environment variables, and configuration management ensure tests work the same locally and in CI.

- Docker for test environments
- Environment configuration
- Test databases
- Service mocking
- Environment parity
- Secrets management

Test Reporting

WHY: Reports communicate quality. Allure, HTML reports, and dashboards help teams understand test results at a glance and track trends over time.

- JUnit/pytest reports
- Allure reports
- Screenshots and videos on failure
- Trend analysis
- Slack/email notifications
- Quality dashboards

Test Selection & Optimization

WHY: Running all tests on every commit doesn't scale. Smart test selection, parallelization, and optimization keep feedback loops fast.

- Smoke vs regression suites
- Test tagging and filtering
- Parallel execution strategies
- Test impact analysis
- Flaky test quarantine
- Test runtime optimization

BDD with Cucumber

WHY: BDD bridges business and tech. Cucumber scenarios become living documentation that stakeholders can read. Many enterprise teams require Gherkin syntax.

- Gherkin syntax (Given-When-Then)
- Feature files
- Step definitions
- Cucumber with Playwright/Selenium
- When to use BDD
- Common BDD mistakes

Hands-on Projects

Complete CI Pipeline

Set up a GitHub Actions pipeline that: runs unit tests, runs API tests, runs UI tests in parallel across browsers, generates Allure reports, and notifies on failure. Make it fast (under 10 minutes).

GitHub Actions

Parallelization

Reporting

Optimization

BDD Feature Suite

Convert an existing test suite to BDD using Cucumber. Write feature files that a product manager could read. Implement step definitions. Run in CI.

Cucumber

Gherkin

BDD principles

Step definitions

Resources

DOCUMENTATION

[GitHub Actions Documentation](#)

DOCUMENTATION

[Allure Framework](#)

DOCUMENTATION

[Cucumber Documentation](#)

Milestone



Have tests running in CI on every PR. Generate reports that stakeholders understand. Pipeline takes under 10 minutes. Zero ignored failures.

7 Specializations

Intensive: Weeks 9-10

Balanced: Weeks 21-26

Flexible: Weeks 41-52

Now you specialize. Performance testing, mobile testing, and security testing are distinct disciplines. This phase introduces each so you can choose your path – or understand enough to collaborate with specialists. SDET roles often require depth in at least one of these areas.

Learning Objectives

- Conduct basic performance tests with k6 or JMeter
- Automate mobile app tests with Appium
- Understand security testing fundamentals
- Know when to involve specialists
- Build a portfolio showcasing your skills

Topics to Cover

Performance Testing

WHY: Fast software that crashes under load isn't quality software. Performance testing finds bottlenecks before users do. Even basic load testing skills are valuable.

- Load vs Stress vs Spike testing
- k6 fundamentals (modern, JavaScript-based)
- JMeter basics (widely used)
- Key metrics (response time, throughput, errors)
- Identifying bottlenecks
- Performance baselines

Mobile Testing with Appium

WHY: Mobile apps need testing too. Appium lets you use familiar tools (WebDriver protocol) for iOS and Android. One codebase can test both platforms.

- Appium architecture
- Setting up for iOS/Android
- Locator strategies for mobile
- Gestures and mobile-specific actions
- Device farms (BrowserStack, Sauce Labs)
- Mobile-specific challenges

Security Testing Basics

WHY: Security bugs are the most expensive. Understanding OWASP Top 10 and basic security testing makes you more valuable – and helps catch vulnerabilities early.

- OWASP Top 10 overview
- SQL injection testing
- XSS testing
- Authentication testing
- Security scanning tools
- When to involve security experts

Building Your Portfolio

WHY: Your GitHub is your resume. A well-documented test framework project demonstrates skills better than any certification. SDET interviews often involve live coding and portfolio review.

- GitHub portfolio best practices
- Documenting test projects
- Open source contributions
- Writing about testing
- Interview preparation
- Demonstrating problem-solving

Hands-on Projects

Performance Test Suite

Use k6 to create a performance test suite for an API. Test baseline performance, simulate increasing load, find the breaking point. Generate and analyze reports. Run in CI.

k6 Performance analysis Load testing
Metrics interpretation

Mobile Automation Framework

Set up Appium and automate tests for a mobile app (use a demo app). Implement POM for mobile. Run on a device farm. Handle mobile-specific scenarios (orientation, gestures).

Appium Mobile testing Device farms
Cross-platform

Complete Test Framework

Build a portfolio-ready test framework from scratch: UI tests, API tests, CI/CD integration, reporting. Document everything. Make it something you'd be proud to show in an interview.

Framework design Documentation
Portfolio building Interview readiness

Resources

DOCUMENTATION [k6 Documentation](#)

DOCUMENTATION [Appium Documentation](#)

GUIDE [OWASP Testing Guide](#)

Milestone



Have depth in one specialization and breadth in others. Build a portfolio project that demonstrates your end-to-end skills. Be ready to discuss testing strategy in SDET interviews.

Your Path Forward

Career Opportunities

✓ QA Engineer

✓ QA Automation Engineer

SDET (Software
✓ Development Engineer in
Test)

✓ Test Architect

✓ QA Lead/Manager

✓ Performance Engineer

Tips for Success

- Developers find bugs; testers prevent them. Shift your mindset from finding to preventing.
- Automate what's repetitive, explore what's complex. Don't automate everything.
- A flaky test is worse than no test – it erodes trust in the entire suite.
- The testing pyramid matters: more unit tests, fewer E2E tests.
- Learn to read code even if you don't write production code.
- Bug reports are documentation. Write them so clearly that anyone could reproduce the issue.
- Understand the business domain. The best testers know the product, not just the code.
- Pair with developers. You'll learn how they think, and they'll learn what you catch.
- Quality is everyone's responsibility, but QA is the advocate.
- Your GitHub portfolio matters more than certifications for SDET roles.

Next Steps

1. Pick a web application and find 5 bugs today
2. Install Postman and explore a public API
3. Choose Python or JavaScript and start learning
4. Set up Playwright and automate your first test
5. Join Ministry of Testing or a QA community
6. Start documenting your learning journey on GitHub

Ready to Start Your Journey?

Everything in this roadmap is free to learn on your own. But if you'd like structured guidance, hands-on projects, and mentor support – we'd love to have you at Crio.Do

Explore Crio.Do

<https://crio.do>